



Prova (a₂)

27 de março de 2026

Nome: _____ Matrícula: _____

1 Instruções

1. Para fazer essa prova, você deve clonar o repositório no seu computador e copiar os arquivos disponibilizados no SIGAA para dentro dele, resolver as questões abaixo e empurrar as soluções para o repositório remoto.
2. Ao clonar o repositório, é preciso instalar as dependências com o comando `npm install`.
3. Cada questão deve ser resolvida no arquivo correspondente, que está localizado no diretório `/questoes`.
4. Apenas os arquivos do diretório `/questoes` podem ser alterados. Não é permitido alterar outros arquivos ou diretórios. A alteração de qualquer arquivo fora do diretório `/questoes` resultará em nota zero.
5. O repositório já está devidamente configurado e não é necessário fazer nenhuma configuração adicional.
6. Você pode testar suas soluções localmente utilizando o comando `npm test`. Esse comando executará todos os testes automatizados.
 - (a) Caso queira executar os testes de uma questão específica, utilize o comando `npm run test:X`, onde `X` é o número da questão (1 ou 2).
 - (b) Caso queira que os testes executem continuamente, adicione `-- --watch` ao final do comando (por exemplo: `npm run test:1 -- --watch` para os testes da questão 1).
7. Não é necessário compilar o TypeScript manualmente, pois os testes já fazem isso automaticamente.
8. É permitida a consulta somente ao material que está na pasta `/docs`. A consulta a qualquer outro material, incluindo a internet, durante a prova resultará em nota zero.
9. Após clonar o repositório da prova, somente uma janela do editor de código (VSCode) pode estar aberta para resolver as questões; a abertura de quaisquer outras janelas do editor de código ou de outros programas durante a prova resultará em nota zero.
10. A detecção de plágio resultará em nota zero para todos os envolvidos.
11. A nota máxima da prova é 10 pontos.

2 Questões

Nesta prova, você ajudará a desenvolver parte de um sistema web de reservas de salas de estudo.

- Na primeira questão, sua tarefa será a de implementar funções capazes de manipular o catálogo de salas exibido e filtrado pela aplicação.
- Na segunda questão, você será responsável por representar as reservas abertas pelos usuários e aplicar as regras de uso e o encerramento dessas reservas. As classes desta questão devem utilizar o mesmo formato de sala definido na questão anterior, de modo que um objeto de sala do catálogo possa ser aproveitado na criação de uma sala reservável.

1. (5 pontos) Utilize **programação funcional em JavaScript** para implementar um sistema simples de gerenciamento de reservas de salas de estudo seguindo os requisitos mínimos (obrigatórios) abaixo. Outras funções podem ser implementadas caso necessário.

Considere que cada sala reservável do sistema tem o seguinte formato:

```
{
  id: number,
  nome: string,
  categoria: "individual" | "grupo" | "laboratorio",
  preco: number,
  quantidade: number,
  disponivel: boolean
}
```

No contexto desta questão, a propriedade `preco` representa o valor por hora da reserva, a propriedade `quantidade` representa a quantidade de horários disponíveis para a sala e a propriedade `disponivel` indica se a sala está liberada para reservas. Esse mesmo formato deve ser compatível com o tipo `Sala` utilizado na segunda questão.

O sistema deve ter as seguintes funcionalidades:

a) Adicionar sala:

- Nome da função: `adicionarSala`
- Parâmetros: `lista` (lista de salas) e `sala` (objeto que representa uma sala)
- Retorno: nova lista incluindo a sala
- Comportamento:
 - se a sala não estiver na lista, adicioná-la;
 - se a sala já existir, não alterar a lista.

b) Remover sala:

- Nome da função: `removerSala`
- Parâmetros: `lista` (lista de salas) e `id` (número do id da sala)
- Retorno: nova lista sem a sala removida
- Comportamento:
 - se a sala não estiver na lista, não alterar a lista;
 - se a sala estiver na lista, removê-la.

c) Atualizar propriedade da sala

- Nome da função: `atualizarPropriedadeSala`
- Parâmetros: `lista` (lista de salas), `id` (número do id da sala), `propriedade` (nome da propriedade a ser atualizada) e `valor` (novo valor da propriedade)
- Retorno: nova lista com a sala atualizada
- Comportamento:
 - se a sala não estiver na lista, não alterar a lista;
 - se a sala estiver na lista, atualizar a propriedade especificada com o novo valor, desde que:
 - * o nome da propriedade esteja correto;
 - * a propriedade não seja `id`, pois o identificador da sala não pode ser alterado;
 - * o valor seja válido para o tipo daquela propriedade (por exemplo, `preco` deve continuar sendo número, `categoria` deve ser um dos valores permitidos).
 - se a propriedade for `id`, se o nome da propriedade ou se o valor não forem válidos, não alterar a lista.

d) Calcular valor total das salas por disponibilidade

- Nome da função: `calcularTotalPorDisponibilidade`
- Parâmetros: `lista` (lista de salas) e `disponibilidade` (valor booleano opcional)
- Retorno: número correspondente à soma de `preco * quantidade` das salas consideradas

- Comportamento:
 - se disponibilidade não for informada, considerar todas as salas;
 - se disponibilidade for true, considerar apenas as salas disponíveis (`disponivel === true`);
 - se disponibilidade for false, considerar apenas as salas indisponíveis (`disponivel === false`);
 - se não houver salas que atendam ao critério, retornar 0.

e) Buscar sala por nome

- Nome da função: `buscarPorNome`
- Parâmetros: `lista` (lista de salas), `nome` (string do nome da sala)
- Retorno: objeto da sala encontrada ou `null`
- Comportamento:
 - buscar a sala que corresponda ao nome fornecido;
 - o nome pode estar em maiúsculas ou minúsculas;
 - se a sala não for encontrada, retornar `null`.

f) Listar resumos de salas

- Nome da função: `listarResumosSalas`
- Parâmetros: `lista` (lista de salas)
- Retorno: lista de strings no formato: "ID: [id], [nome] ([categoria]) - R\$ [valor total: preco x quantidade] - disponível: [sim/não]"
- Comportamento:
 - Caso a sala esteja com `disponivel === true`, o texto *disponível* deve ser "sim";
 - Caso contrário, deve ser "não".

g) Limpar salas indisponíveis

- Nome da função: `limparIndisponiveis`
- Parâmetros: `lista` (lista de salas)
- Retorno: nova lista apenas com as salas que ainda estão disponíveis para reserva.

2. (5 pontos) Utilize **orientação a objetos em TypeScript** para modelar o módulo de reservas do mesmo sistema web de salas de estudo seguindo os requisitos mínimos (obrigatórios) abaixo. Outros métodos e propriedades podem ser adicionados conforme necessário.

a) A interface `Sala`, que representa o formato comum de sala utilizado pelas duas questões:

- Propriedades:
 - `id` (number);
 - `nome` (string);
 - `categoria` ("individual" | "grupo" | "laboratorio");
 - `preco` (number);
 - `quantidade` (number);
 - `disponivel` (boolean).
- Essa interface deve representar exatamente o mesmo formato de objeto manipulado pelas funções da primeira questão.

b) Uma classe abstrata `SalaReservavel`:

- Propriedades (protegidas):
 - `id` (number): identificador da sala;
 - `nome` (string): nome da sala;
 - `categoria` ("individual" | "grupo" | "laboratorio"): categoria da sala;
 - `preco` (number): valor por hora da reserva;
 - `quantidade` (number): quantidade de horários disponíveis;
 - `disponivel` (boolean): indica se a sala está liberada para reservas;
 - `reservas` (lista de objetos do tipo `{ usuario: Usuario }`): registra as reservas atuais da sala.

- Construtor: deve receber um objeto do tipo `Sala` e inicializar as propriedades da classe.
- Métodos:
 - `descricao`
 - * Parâmetros: nenhum;
 - * Retorno: `string`;
 - * Comportamento: retorna uma descrição da sala no formato: "ID: [id], [nome] ([categoria]) - R\$ [preco] por hora - horários disponíveis: [quantidade] - disponível: [sim/não]"
 - `reservar`
 - * Parâmetros: `usuario: Usuario`;
 - * Retorno: `boolean` indicando se a operação foi bem-sucedida;
 - * Comportamento:
 - permite reservar se a sala estiver disponível e houver horários disponíveis;
 - decrementa o valor de `quantidade` em caso de sucesso;
 - define `disponivel` como `false` se `quantidade` for igual a 0 após a reserva;
 - registra a reserva com o usuário informado;
 - um mesmo usuário não pode reservar a mesma sala mais de uma vez simultaneamente (não pode ter duas reservas abertas da mesma sala).
 - `encerrarReserva` (abstrato)
 - * Parâmetros: `usuario: Usuario`;
 - * Retorno: `number`.
 - `toSala`
 - * Parâmetros: nenhum;
 - * Retorno: `Sala`;
 - * Comportamento: retorna um objeto no mesmo formato da interface `Sala`, refletindo o estado atual da sala reservável, especialmente os valores atualizados de `quantidade` e `disponivel`.

c) A classe `SalaGrupo` que estende `SalaReservavel`:

- Propriedade (privada):
 - `recursos` (lista de objetos do tipo `Recurso`).
- Construtor: deve receber um objeto do tipo `Sala` com `categoria` igual a "grupo" e inicializar as propriedades herdadas e a propriedade da classe.
- Métodos:
 - Implementar o método `encerrarReserva`
 - * Retorno: 0 se a reserva for encerrada com sucesso ou -1 para indicar erro;
 - * Comportamento: encerra a reserva aberta do usuário informado, remove essa reserva da lista de reservas da sala e incrementa o valor de `quantidade`;
 - * define `disponivel` como `true` se `quantidade` for maior que 0 após o encerramento da reserva;
 - * observação: caso o usuário não tenha reservado a sala de grupo, o método deve retornar -1 para indicar erro. Não é necessário calcular multa nem consultar a hora atual neste método.
 - Sobrescrever o método `descricao`
 - * Retorno: `string`;
 - * Comportamento: adicionar ao final da descrição herdada, separado por vírgula, o texto: "Recursos: [nome] [tipo] ([localizacao])"
 - * Todos os recursos da lista `recursos` devem ser listados, separados por ponto e vírgula.

d) A classe `Usuario`, que representa um usuário do sistema de reservas:

- Propriedades (privadas):
 - `nome` (`string`): nome do usuário;
 - `codigo` (`number`): código do usuário, que deve ser único;

- `codigos` (`number []` – estática): lista de códigos já utilizados para garantir unicidade.
- Construtor: deve inicializar as propriedades da classe, garantindo que `codigo` seja único.
- Métodos:
 - getters para `nome` e `codigo`;
 - getter estático para `codigos`;
 - `gerarCodigo` (estático)
 - * Parâmetros: nenhum;
 - * Retorno: número;
 - * Comportamento: gera um código único para um novo usuário.

e) A interface `Recurso` com as seguintes propriedades:

- `nome (string)`;
- `tipo (string)`;
- `localizacao (string)`.